



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A KOMUNIKAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV TELEKOMUNIKACÍ

DEPARTMENT OF TELECOMMUNICATIONS

MULTIPLATFORMNÍ APLIKACE PRO HROMADNOU SPRÁVU ZAŘÍZENÍ MIKROTIK ROUTEROS

MULTI-PLATFORM APPLICATION FOR BULK MANAGEMENT OF MIKROTIK ROUTEROS DEVICES

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

Jiří Kozárek

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. Ondřej Krajsa, Ph.D.

BRNO 2026

Bakalářská práce

bakalářský studijní program **Telekomunikační a informační systémy**

Ústav telekomunikací

Student: Jiří Kozárek

ID: 256766

Ročník: 3

Akademický rok: 2025/26

NÁZEV TÉMATU:

Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS

POKYNY PRO VYPRACOVÁNÍ:

Navrhněte a implementujte multiplatformní aplikaci (Windows, Linux) v Pythonu pro správu zařízení s RouterOS prostřednictvím oficiálního API (šifrované připojení TLS na portu 8729). Aplikace musí umožnit kompletní správu jednotlivých zařízení, bezpečné uložení přístupových údajů, jednotlivé i hromadné provádění konfiguračních změn (např. přes šablony), zálohu a obnovu konfigurace a jednoduchou správu logů. Výstupem práce bude plně funkční aplikace, zdrojové kódy a uživatelská a instalační příručka.

DOPORUČENÁ LITERATURA:

Podle pokynů vedoucího práce.

Termín zadání: 9.2.2026

Termín odevzdání: 2.6.2026

Vedoucí práce: Ing. Ondřej Krajsa, Ph.D.

prof. Ing. Jiří Mišurec, CSc.
předseda rady studijního programu

UPOZORNĚNÍ:

Autor bakalářské práce nesmí při vytváření bakalářské práce porušit autorská práva třetích osob, zejména nesmí zasahovat nedovoleným způsobem do cizích autorských práv osobnostních a musí si být plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

ABSTRAKT

Bakalářská práce má za cíl vytvořit multiplatformní aplikaci v programovacím jazyce Python, jež bude schopna provádět hromadné konfigurace aktivních síťových prvků, na kterých běží operační systém MikroTik RouterOS. Konfigurace bude probíhat z grafického rozhraní pomocí systémového rozhraní API na portu 8729, konkrétně jeho šifrované verze zabezpečené pomocí TLS. Celá aplikace je dostupná v repozitáři na platformě GitHub.

KLÍČOVÁ SLOVA

Python, MikroTik, RouterOS, hromadná konfigurace

ABSTRACT

The goal of this bachelor's thesis is creation of cross-platform application written in Python programming language that will be capable of mass configuration of active network devices running MikroTik RouterOS operating system. The configuration will be done via graphical user interface using system API interface on port 8729, specifically its encrypted version secured by TLS. The entire app is available in a repository on GitHub platform.

KEYWORDS

Python, MikroTik, RouterOS, mass configuration

KOZÁREK, Jiří. *Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS*. Bakalářská práce. Brno: Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav telekomunikací, 2026. Vedoucí práce: Ing. Ondřej Krajsa, Ph.D.

Prohlášení autora o původnosti díla

Jméno a příjmení autora:	Jiří Kozárek
VUT ID autora:	256766
Typ práce:	Bakalářská práce
Akademický rok:	2025/26
Téma závěrečné práce:	Multiplatformní aplikace pro hromadnou správu zařízení MikroTik RouterOS

Prohlašuji, že svou závěrečnou práci jsem vypracoval samostatně pod vedením vedoucí/ho závěrečné práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury na konci práce. V souvislosti s vytvořením závěrečné práce jsem neporušil zásady a doporučení VUT k využívání generativní umělé inteligence.

Jako autor uvedené závěrečné práce dále prohlašuji, že v souvislosti s vytvořením této závěrečné práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a/nebo majetkových a jsem si plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

Brno
podpis autora*

*Autor podepisuje pouze v tištěné verzi.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu bakalářské práce panu Ing. Ondřeji Krajsovi, Ph.D., za vedení, konzultace a podnětné návrhy k práci. Dále bych chtěl také poděkovat panu Bc. Jakubu Raimrovi za rady při psaní této práce a panu Martinu Smejkalovi za věcné rady v oblasti zabezpečení dat, bezpečnosti a programování.

Obsah

Úvod	11
1 Síťová zařízení	12
1.1 MikroTik	12
2 RouterOS	13
2.1 Konfigurace	13
2.2 winbox/webfig	13
2.3 API	15
3 Návrh aplikace	16
3.1 Databáze	16
3.1.1 Možné realizace databáze	16
3.1.2 Ukládání hesel	17
3.2 Grafické rozhraní	18
3.3 Pohledy	18
3.4 Bezpečnost aplikace	19
3.5 Hromadné operace	20
4 Implementace	21
4.1 Komunikace se systémem RouterOS pomocí API	21
4.2 Databáze	22
4.2.1 Databáze zařízení	23
4.2.2 Databáze příkazů	24
4.3 GUI	25
5 Testování	27
5.1 Ošetřené stavy	27
6 Struktura aplikace	29
6.1 Úvodní aplikace	29
6.1.1 Vnitřní struktura	30
6.2 Konfigurační aplikace	31
6.2.1 Vnitřní struktura	34
6.3 Administrátorská aplikace	36
Závěr	38
Literatura	39

Seznam symbolů a zkratk	41
A Aplikace micoto	42

Seznam obrázků

2.1	Některé z možností konfigurace ROS	13
2.2	Ukázka grafického rozhraní WinBox	14
2.3	Ukázka webového rozhraní WebFig	14
3.1	Jednoduchý ilustrační diagram interakce jednotlivých částí a aplikací mezi sebou	16
3.2	Grafické rozhraní konfiguračního okna (vývojová verze)	18
3.3	Diagram návrhu realizace hromadné konfigurace	20
4.1	Diagram realizace oddělených databázových souborů	23
4.2	Diagram realizace databázové tabulky obsahující záznamy zařízení . .	23
4.3	Diagram realizace databázových tabulek obsahujících záznamy kon- figuračních cest a příkazů	24
5.1	Ukázka chybového okna	28
6.1	Grafické rozhraní hlavní aplikace	29
6.2	Grafické rozhraní konfigurační aplikace	32
6.3	Detail grafického rozhraní konfigurační aplikace - sekce <i>common</i> . . .	33
6.4	Detail grafického rozhraní konfigurační aplikace - záznamy vybraného zařízení	33
6.5	Detail grafického rozhraní konfigurační aplikace - konfigurační podokno	33
6.6	Detail grafického rozhraní aplikace - administrátorský mód	36
6.7	Detail grafického rozhraní aplikace - rozvinuté administrátorské menu	36
6.8	Detail grafického rozhraní aplikace - okno <i>treeview</i>	37
6.9	Detail grafického rozhraní aplikace - okno <i>treeview</i> s otevřenými mož- nostmi výběru větve	37

Seznam výpisů

2.1	Ukázka komunikace pomocí api s využitím ukázkového skriptu [11], zkráceno	15
4.1	Ukázka přihlášení při komunikaci s rozhraním API	21
4.2	Ukázka inicializace konfiguračního okna, zkráceno	25
6.1	Metoda objektu <i>db_crypto</i> (6.1.1) pro zašifrování a uložení přístupových údajů	31
6.2	Metoda objektu <i>middleware</i> (6.2.1) sloužící k provedení konfiguračních změn	35

Úvod

Bakalářská práce se zaměřuje na návrh a realizaci multiplatformní aplikace pro hromadnou správu zařízení s operačním systémem RouterOS společnosti MikroTik. Pro komunikaci se zařízeními má aplikace využívat zabezpečenou komunikaci s rozhraním API na portu 8729.

První a druhá kapitola zběžně uvádí čtenáře do tématu síťových prvků a společnosti MikroTik. Dále pojednávají o systému RouterOS společnosti MikroTik. Uvádí vybrané způsoby jeho konfigurace se zaměřením na aplikaci WinBox a rozhraní API. Třetí kapitola řeší návrh architektury aplikace. Řeší problematiku databáze. Probírá její strukturu i možnosti realizace. Dále také pojednává o způsobu uložení přihlašovacích údajů. Nabízí také návrh grafického rozhraní aplikace. Popisuje systém pohledů. Pojednává také o bezpečnosti aplikace a návrhu implementace provádění hromadných operací.

Čtvrtá kapitola se zaměřuje na samotnou implementaci jednotlivých částí aplikace. Popisuje detaily jednotlivých částí.

Pátá kapitola stručně představuje provedené testy aplikace.

Šestá kapitola popisuje běh aplikace a jejich jednotlivých částí.

1 Síťová zařízení

Dnešní svět, silně propojený celosvětovou počítačovou sítí Internet, si bez její existence již nedokážeme představit. Díky této síti jsme schopni docílit téměř real-time komunikace a to z jakéhokoli místa na světě na jiné. Vznik takovéto sítě je podmíněn vysokým stupněm standardizace [1]. Síť, skládající se z pasivních a aktivních částí, vyžaduje správu.

1.1 MikroTik

Na světě existuje nespočet firem, malých či velkých, které se zabývají vývojem nebo výrobou speciálního softwaru či hardwaru pro síťové aplikace. Namátkou lze vybrat firmy známé, jejichž jména se stala v oboru ikonickými, jako jsou společnosti CISCO Systems [2], HP (Hewlett-Packard) [3] a její odnož Enterprise [4], či Arista [5]. Z řady menších, ale stále dosti významných firem, stojí za zmínku firma Ubiquiti [6], která se snaží o zjednodušení konfigurace a správy aktivních síťových prvků. Zaměřením této práce je však litevská firma SIA Mikrotīks, známá jako MikroTik [7]. Jedná se o výrobce kompletních síťových řešení v oblasti aktivních prvků ve smyslu výroby vlastního síťového hardwaru [8] a především síťového softwaru RouterOS [9].

2 RouterOS

Síťový operační systém ROS společnosti MikroTik je komunitou vysoce oblíbený a rozšířený napříč spektrem síťových zařízení. Jeho distribuce probíhá společně s hardwarovými produkty. Díky oblibě komunity se také stal samostatným produktem společnosti. Lze jej pořídit samostatně a provozovat přímo na strojích s architekturou x86 či využít verzi CHR a provozovat tento systém jako virtuální stroj. Tímto operačním systémem je nabízena konfigurace nepřeberného množství protokolů převážně na prvních třech vrstvách ISO/OSI modelu [10].

2.1 Konfigurace

Systémem ROS je nabízeno velké množství způsobů konfigurace. K navigaci je použita stromová struktura příkazů, ve které je třeba se pro podrobnější správu vydat po dané větvi dál od konfiguračního kmene až k samotným možným argumentům příkazů. Na následujícím obrázku je zobrazen seznam možností přístupů konfigurace a port, na kterém služba aktuálně naslouchá. Stojí za zmínku, že všechny zobrazené služby běží na standardních (výchozích) portech.

Name	Port
api	8728
api-ssl	8729
ftp	21
ssh	22
telnet	23
winbox	8291
www	80
www-ssl	443

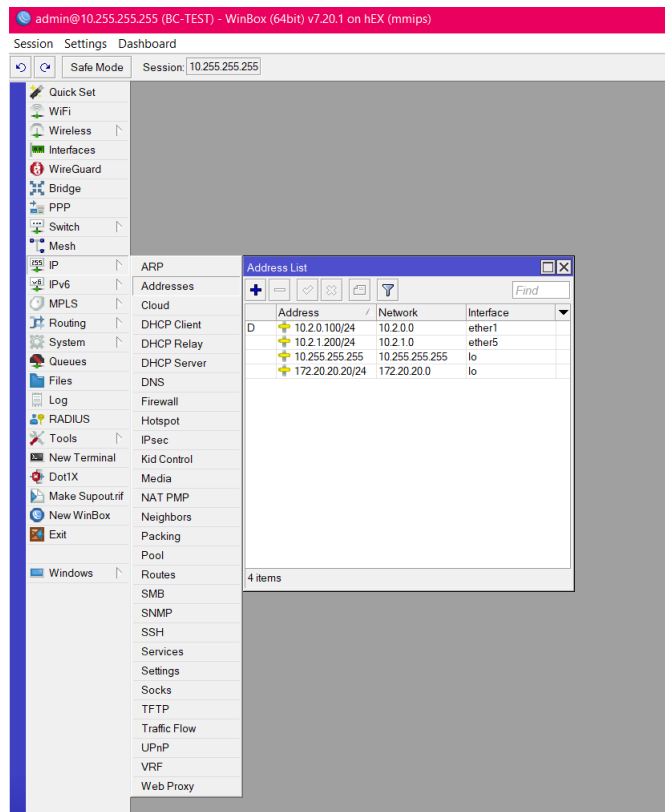
Obr. 2.1: Některé z možností konfigurace ROS

Pro účely této práce jsou však jiné způsoby konfigurace nežli *winbox/webfig* a *api* bezvýznamné, proto jim již nebude v této práci věnována větší pozornost.

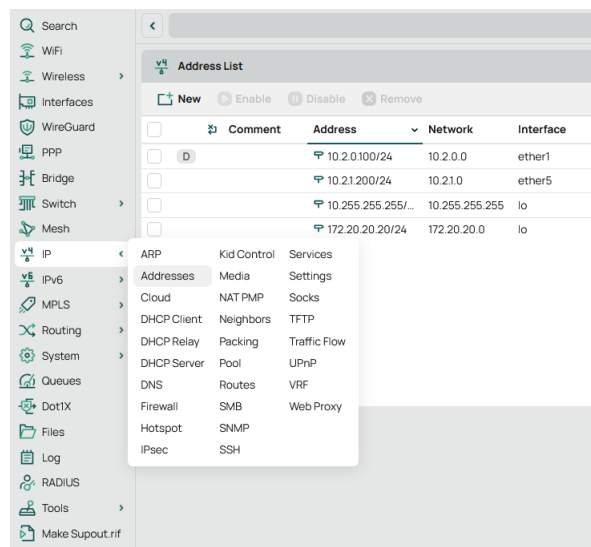
2.2 winbox/webfig

WinBox je grafický proprietární konfigurační software pro správu zařízení se systémem Mikrotik RouterOS. Je založen na systému oken s konfiguračními podokny, pomocí kterých se správa provádí. Lze konstatovat, že se jedná o grafické zobrazení stromové hierarchie příkazů dostupných v konzoli. Stejně tak lze konstatovat, že konfigurační rozhraní webfig plní funkci winboxu. Jedná se však o webovou aplikaci vzhledově a do jisté míry i funkčně podobnou aplikaci winbox. Její největší výhoda

spočívá v možnosti konfigurace prakticky z každého zařízení, na kterém lze spustit webový prohlížeč.



Obr. 2.2: Ukázka grafického rozhraní WinBox



Obr. 2.3: Ukázka webového rozhraní WebFig

2.3 API

Jedná se o přímé napojení využívající komunikaci na úrovni bytů, kterou je třeba dekodovat. Samotná užitečná komunikace pak probíhá ve formátu příkazového slova společně s jeho argumenty, následuje odpověď daného zařízení ve formě výčtu ve formátu parametr – hodnota.

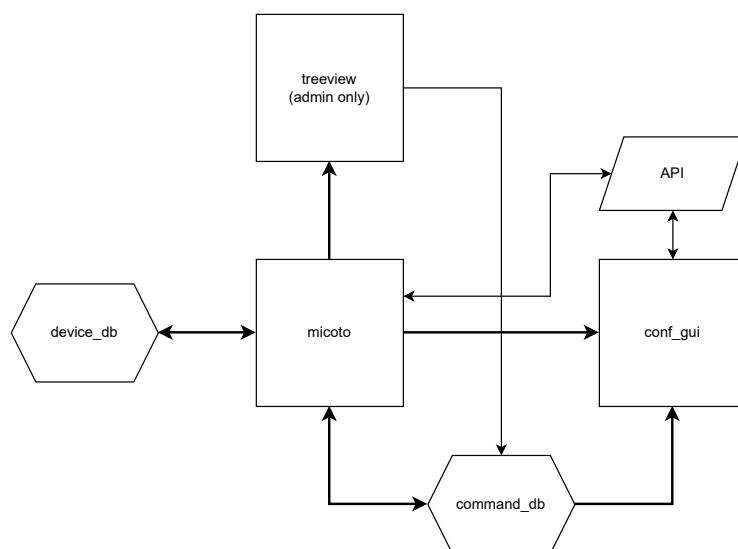
Výpis 2.1: Ukázka komunikace pomocí api s využitím ukázkového skriptu [11], zkráceno

```
1 <<< /ip/address/print
2 <<<
3 >>> !re
4 >>> =.id=*2
5 >>> =address=10.255.255.255/32
6 >>> =network=10.255.255.255
7 >>> =interface=lo
8 >>> =actual-interface=lo
9 >>> =invalid=false
10 >>> =dynamic=false
11 >>> =slave=false
12 >>> =disabled=false
```

Ukázka nevyužívá všechna možná slova a parametry, které lze použít při komunikaci. Pro ukázkou je tento jednoduchý příkaz dostatečný. Dále je také v práci uvažována komunikace s api rozhraním v zabezpečené formě využívající TLS - v ukázce pod názvem *api-ssl*.

3 Návrh aplikace

Aplikace je rozdělena na několik samostatně funkčních bloků, a to samotnou aplikaci *micoto* v uživatelském či administrátorském režimu, aplikaci *treeview* sloužící ke grafické reprezentaci stromu příkazů systému RouterOS a jejich výběru a uložení do databáze a konfigurační aplikaci *conf_gui* sloužící k realizaci samotných konfiguračních změn.



Obr. 3.1: Jednoduchý ilustrační diagram interakce jednotlivých částí a aplikací mezi sebou

3.1 Databáze

Jedním ze základních úkolů této práce pro autora byl návrh databáze a systému ukládání hesel. Struktura databáze má dalekosáhlé důsledky pro návrh celé architektury aplikace a její budoucí možnosti.

3.1.1 Možné realizace databáze

Autor vybíral ze základních dvou přístupů pro práci s databází. Jedním z nich je přístup pomocí serveru. V tomto modelu je třeba serverová část, spuštěná lokálně či na vzdáleném serveru, ke které se uživatel připojí a využívá její služby pomocí síťového stacku operačního systému. Má však velké úskalí, díky kterému je dle autora nevhodná pro tuto aplikaci.

Při lokálně spuštěné serverové instanci je nejprve nutno tuto instanci na daném počítači nainstalovat a spustit. Takové řešení sice je možné, ale autor má za to, že takové řešení přidává zbytečnou komplexitu projektu.

Při použití vzdáleného serveru je předpokládána funkčnost síťového spojení na daný server a jeho funkčnost. Z logiky vyplývá, že by zde mohl nastat zásadní problém, a to v momentu výpadku. Nastala by situace, kdy by síťové prvky, nutné pro spojení s databázovým serverem, nemohly být nakonfigurovány, jelikož přístupové údaje k těmto prvkům by byly uloženy na v tu chvíli nedostupném serveru. Z výše uvedeného vyplývají zásadní možné chyby a problémy dané realizace databáze, proto autor vybral alternativní metodu uvedenou níže.

SQLite

Díky předešlým úvahám a vlastnostem uvedených systémů autor využil v aplikaci SQLite [12], konkrétně implementaci tohoto systému knihovnou sqlite3 [13]. Jedná se o lightweight knihovnu obsaženou přímo v Pythonu, není tedy třeba její doinstalace. Tento systém nevyužívá spojení na databázový server, nýbrž lokální databázový soubor. Aplikace tedy bude fungovat i v případě kompletního odpojení od sítě či jejím výpadku. Je třeba zmínit hlavní nevýhodu tohoto přístupu, a tou je nutnost databázové soubory distribuovat či synchronizovat. Dále také fakt, že nemůže být přistupováno k databázovému souboru vícero uživateli najednou. Při využívání databázového souboru je soubor uzamčen pro čtení i zápis ostatním uživatelům. Potenciální problémy s přístupem k databázovému souboru však nejsou pro tuto aplikaci relevantní, jelikož aplikaci může využívat v daném okamžiku na jednom počítači pouze jeden uživatel. Otázka distribuce a synchronizace bude popsána dále. Proto má autor za to, že řešení pomocí systému SQLite, používající lokální databázové soubory, bude nejvhodnějším řešením.

3.1.2 Ukládání hesel

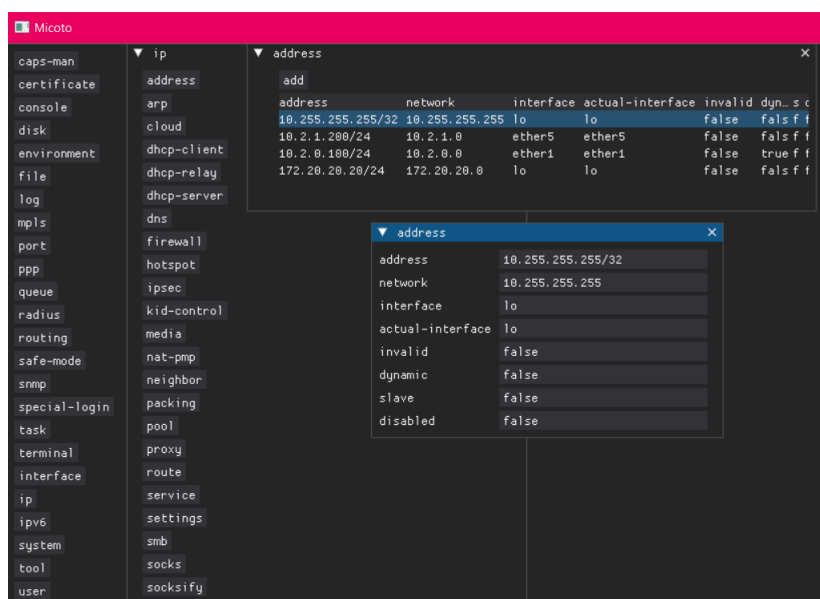
Ačkoli je autorovi známo, že ukládání hesel jako takových je naprosto nevhodné a silně nedoporučené, v tomto případě není zbytlí. Systém ROS nepodporuje pro přístup pomocí api jinou autentizační metodu nežli přihlášení uživatelským jménem a heslem. Jelikož je jeden pohled aplikace zamýšlen spíše uživatelsky, viz dále, je nutné vytvořit jednoduchý systém, kdy po zadání jednoho hesla budou zpřístupněna všechna zařízení. Protože nastavení stejného hesla do vícero zařízení není z bezpečnostního hlediska lepším řešením, zvolil autor řešení využívající databázi se šifrovanými přístupovými hesly.

Zprvu bylo autorem zamýšleno využít systém symetrického šifrování s využitím

knihovny cryptography, a to konkrétně již připraveným systémem Fernet [14]. Tento systém je jednoduchý na implementaci, avšak po konzultaci s vedoucím práce bylo rozhodnuto pro jeho nevyužití, jelikož pro šifrování je využíván AES-128-CBC. Ačkoli je AES-128 dnes stále považován za bezpečný a splňuje minimální požadavky NÚKIBu [15], bylo rozhodnuto o nevyužití tohoto systému a implementaci řešení využívající AES-256-CBC, na který je aktuálně nahlíženo jako i do budoucna bezpečný algoritmus.

3.2 Grafické rozhraní

Práce s aplikací má probíhat pomocí GUI. Programovací jazyk Python nabízí nemalé množství knihoven, které implementaci grafických rozhraní ulehčují [16]. Jelikož je předpoklad funkční podobnosti aplikace s aplikací WinBox, bylo rozhodnuto využít knihovnu DearPyGui [17] [18]. Jedná se o lightweight grafickou knihovnu s hardwarovou akcelerací. Hlavním důvodem výběru této knihovny je možnost jednoduché práce s okny v okně. Předpoklad této funkce je stěžejní pro celou koncepci programu. Stejně tak je nutné, aby grafické rozhraní bylo multiplatformní.



Obr. 3.2: Grafické rozhraní konfiguračního okna (vývojová verze)

3.3 Pohledy

Spousta systémů síťových prvků nabízí možnost využít tzv. „views“ či „role-based“ přístup [19]. Tato funkcionalita umožňuje definovat, k jakým příkazům a jejich para-

metrům bude mít daný uživatel systému přístup. Právě tato funkcionality v systému ROS chybí. Díky výše zmíněné volbě architektury (4.2) využívající lokální databázi dostupných příkazů lze vytvořit pro daného uživatele speciální databázový soubor obsahující pouze příkazy a parametry, které potřebuje ke své práci, a to za účelem zjednodušení či jako „bezpečnostní“ faktor, kdy není uživatel schopen jednoduše procházet či měnit části konfigurace z aplikace, s jejichž fungováním není plně seznámen a/nebo k nim nemá mít přístup.

3.4 Bezpečnost aplikace

Aplikace je navržena s nemalým ohledem na bezpečnost. I přes tento fakt je zde prostor pro zlepšení.

Hlavní oblastí ke zlepšení je oblast zabezpečení databáze. Nejedná se tak o zabezpečení uložených dat, ale spíše slabého místa architektury, a tou je heslo. Na hesla jsou z bezpečnostního hlediska kladeny nároky jako je velká délka či komplexita, tyto požadavky však často nejsou v reálném světě reflektovány a v případě jejich vynucování může docházet k případům, kdy si uživatelé svá hesla zapisují v čitelné formě elektronicky či na papírky a bloky umístěné poblíž pracoviště či přímo na nich. Hesla jsou dlouhodobě známá jako nejslabší článek jakéhokoli bezpečnostního řetězce, a to především díky tomu, že uživatelé si nastavují hesla krátká a pro ně srozumitelná, jsou tedy náchylná na slovníkové útoky. Právě v krádeži databáze a slovníkovém útoku tkví největší bezpečnostní hrozba tohoto systému.

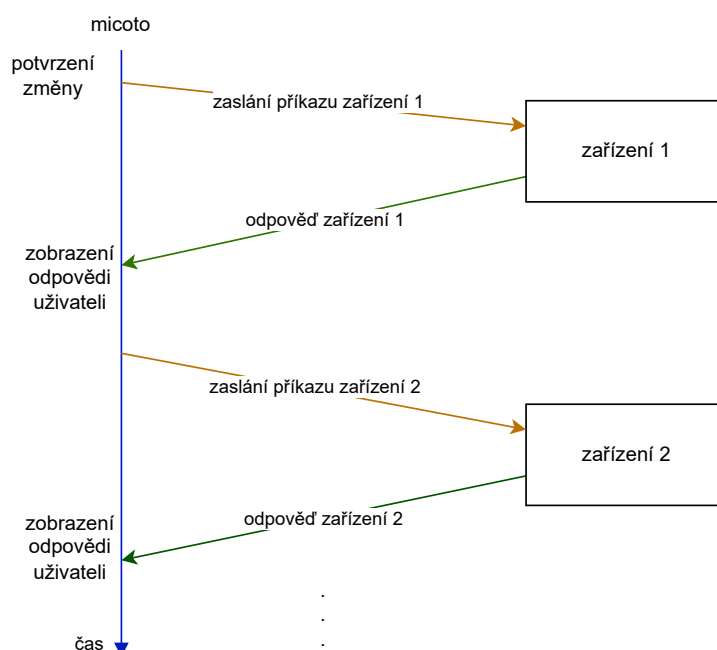
Jedním z možných řešení tohoto stavu je odstranění hesel a realizace ověření uživatele pomocí uživatelských certifikátů vydaných důvěryhodnou lokální certifikační autoritou, pro šifrování pak může být použito dlouhé a komplexní heslo, což snižuje riziko při použití slovníkového útoku.

Jako další možné řešení je přesun aplikace na server. Při tomto řešení se z aplikace stane centralizované řešení mající své nevýhody spočívající v nutnosti funkčního spojení klient–server a server–konfigurované zařízení. Toto řešení však výrazně snižuje riziko krádeže databáze a zároveň umožňuje využít silný šifrovací klíč a snížit tak riziko prolomení hesla slovníkovým útokem. Zároveň by toto řešení mohlo umožňovat napojení na centrální systém řízení uživatelů ve smyslu Active Directory/LDAP, uložení dat do jedné centrální databáze a vyšším možností kontroly nad prováděnými akcemi.

3.5 Hromadné operace

Provádění hromadných operací ve smyslu úpravy konfigurace vícero zařízení současně není od výrobce implementováno v žádné formě, proto bylo třeba vymyslet kompletně novou logiku pro řízení této funkcionality.

Pro řízení byl navrhnut kvůli jednoduchosti implementace systém postupného odesílání příkazů jednotlivým zařízením. Po potvrzení konfiguračních změn uživatelem dojde k postupnému aplikování na všech zařízeních, kterých se dané změny týkají. Po příchodu každé odpovědi je uživatel informován o tom, jaký je výsledek požadované operace (odpověď zařízení).



Obr. 3.3: Diagram návrhu realizace hromadné konfigurace

4 Implementace

Aplikace je rozdělena do několika samostatně funkčních celků, a to administrátorské aplikace, samotné konfigurační aplikace a úvodní aplikace, která je otevřena ihned po spuštění aplikace micoto.

Co se databází týče, je aplikace rozdělena na dvě části. Tyto části jsou databáze s příkazy a databáze s přihlašovacími údaji. Toto rozdělení má své výhody i nevýhody.

Pro dané rozdělení databází bylo rozhodnuto kvůli praktičnosti reálného nasazení. Administrátorovi stačí pro změnu dostupných příkazů či změně dostupných zařízení rozdistribuovat nové databázové soubory s definovanou strukturou, a to nezávisle na ostatních uživateli. Tato skutečnost také umožňuje tyto dvě databáze upravovat nezávisle na sobě a tvořit jejich kombinace.

Aplikace využívá jazyk *Python3*, pro grafické rozhraní knihovnu *dearpygui* a pro práci se databází knihovnu *sqlite3*. Šifrování dat je realizováno pomocí algoritmu AES-256-CBC.

4.1 Komunikace se systémem RouterOS pomocí API

Jak již bylo zmíněno, komunikace probíhá pomocí zabezpečené verze rozhraní *API* na portu 8729. Rozhraní *API* realizuje komunikaci v textové formě. Způsob komunikace bude představen na následujícím příkladu, ve kterém dojde k přihlášení a výpisu ip adres na zařízení.

Jako první krok pro navázání komunikace je otevření spojení, socketu, se zařízením a následné šifrování obsahu pomocí TLS. Ihned po tomto navázání síťového spojení je nutné provést přihlášení.

Výpis 4.1: Ukázka přihlášení při komunikaci s rozhraním API

```
1 <<< /login
2 <<< =name=admin
3 <<< =password=password
4 <<<
5 >>> !done
6 >>>
```

Znaky `<` a `>` v ukázce slouží pouze pro naznačení směru komunikace, v reálné komunikaci se vůbec nevyskytují. Pro zadávání požadavků slouží dva typy slov.

První z těchto slov je slovo typu *příkaz*. Jedná se o klíčová slova systému vykonávající nějakou funkci. Jedná se buďto přímo o dané slovo případně cestu k příkazu,

který je dále od kořene stromu. Cesta je určena pomocí lomítkové notace, příklad */ip/address/print*. V jedné větě, sdružení vícero slov různého typu, může být pouze jedno slovo typu *příkaz* a vícero slov typu *atribut*. Pro oddělení jednotlivých slov slouží znak zalomení řádku. Příkladem slova typu *příkaz* v ukázce je */login*.

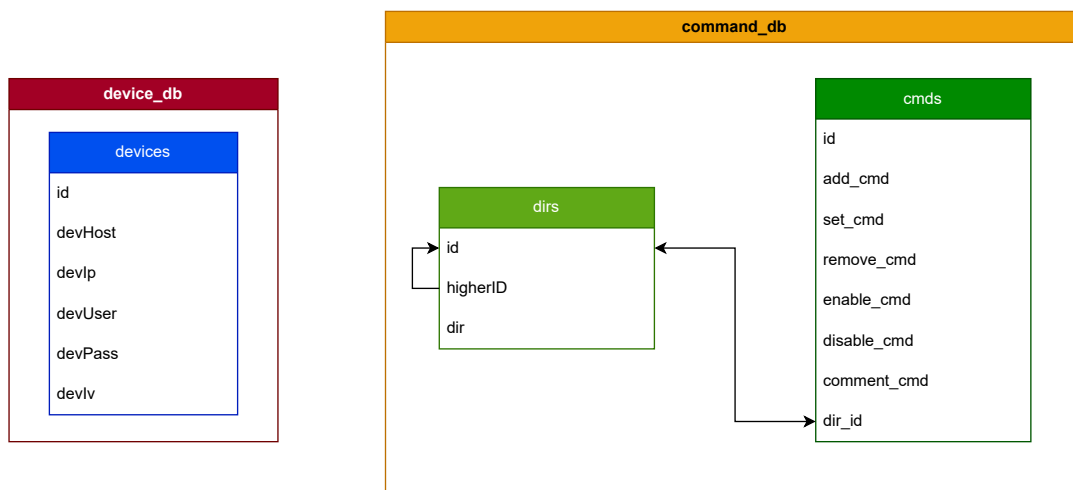
Za slovem typu *příkaz* následuje slovo typu *atribut*. Toto slovo nese informaci o názvu požadovaného parametru a nastavuje jeho hodnotu. Formát tohoto slova je *=nazev_atributu=hodnota_atributu*. Stejně jako dostupné *příkazy*, tak i dostupné *atributy* jsou závislé na aktuálním umístění ve stromu příkazů. Příkladem slov typu *příkaz* v ukázce je *=name=admin* a *=password=password*.

Následující prázdný řádek není náhodný. Systém API pro indikaci ukončení věty využívá prázdný znak. Proto se po každé, ať už přijaté či odeslané větě, v ukázce nachází prázdný řádek.

Následuje odpověď od zařízení *!done*. Znak *!* identifikuje návratový kód odeslaného příkazu. Mezi návratové kódy patří *!re* značící bezchybné provedení příkazu, *!done* úspěšné aplikování změn. Nejpodstatnější je však kód *!trap* značící selhání vykonávání příkazu. Za tímto kódem následuje atribut *category* s číselnou hodnotou jež reprezentuje důvod selhání a atribut *message* s bližším textovým popisem chyby.

4.2 Databáze

Databáze je rozdělena na dvě hlavní části, a to tabulka *devices* obsahující záznamy týkající se zařízení a tabulky *dirs* a *cmds* uchováující informaci o stromu příkazů, respektive o dostupnosti vybraných příkazů (4.1). Díky rozdělené architektuře zařízení a dostupných příkazů je možné distribuovat jednotlivé části dohromady v jednom databázovém souboru či samostatně. Každá z možností má své výhody a nevýhody. První zmíněná, tedy využití jednoho databázového souboru, s sebou přináší jednoduchost co se distribuce týče, hlavní nevýhodou je však složitost úpravy jedné z částí, například přidání či odebrání zařízení. V takovém případě je totiž nutné vytvářet všechny kombinace seznamu zařízení a příkazů znovu. Velkého zjednodušení je možné dosáhnout úpravou stávajícího databázového souboru obsahujícího všechny tabulky. Takové řešení sice zjednoduší implementaci změn, ale nezabrání stavu, kdy je třeba takové změny provést pro velké množství souborů. Jako druhá možnost je systém oddělených databázových souborů se zařízeními a příkazy. Hlavní výhodou tohoto řešení je modularita, kdy stačí upravit pouze jeden z vybraných databázových souborů a rozdistribuovat takto upravený soubor větší skupině lidí. V případě využití sdílených prostředků ve smyslu sdíleného úložiště lze takto učinit bez zásahu uživatelů prostou záměnou daného souboru za nový se stejným názvem. Aplikace micoto podporuje oba dva přístupy.



Obr. 4.1: Diagram realizace oddělených databázových souborů

4.2.1 Databáze zařízení

Pro práci s databází zařízení vznikla třída *DBConn*. Tato třída zajišťuje vytváření databázového souboru a tabulky, přidávání a mazání záznamů zařízení. Zároveň tato třída implementuje mechanismus šifrování, a to díky knihovně cryptography, konkrétně využívá AES-256-CBC. Ke generování inicializačního vektoru je využita funkce *random* z knihovny *os*.

devices	
id	INT, PK
devHost	VARCHAR(255)
devIp	VARCHAR(255)
devUser	VARCHAR(255)
devPass	VARCHAR(255)
devIv	VARCHAR(255)

Obr. 4.2: Diagram realizace databázové tabulky obsahující záznamy zařízení

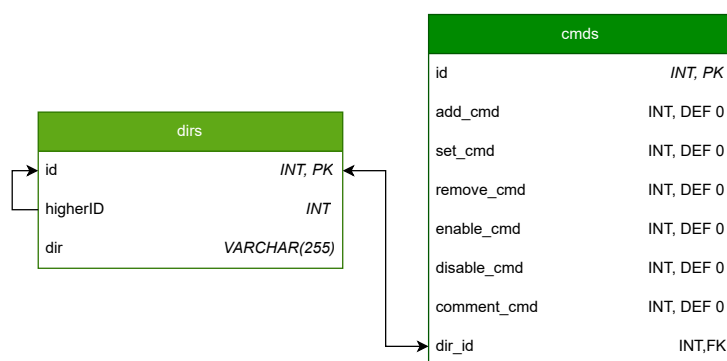
Způsob ukládání zařízení je velice jednoduchý a funkční. Všechny informace jsou uloženy do jedné tabulky. Tabulka obsahuje:

- *id* - jednoznačný číselný automaticky generovaný identifikátor sloužící jako primární klíč,
- *devHost* - hodnota typu VARCHAR uchovávající jméno zařízení,
- *devIp* - hodnota typu VARCHAR uchovávající IP adresu zařízení,
- *devUser* - hodnota typu VARCHAR uchovávající uživatelské jméno,

- *devIp* - hodnota typu VARCHAR uchovávající heslo,
- *devIp* - hodnota typu VARCHAR uchovávající inicializační vektor sloužící jako jeden ze vstupů pro správné dešifrování.

4.2.2 Databáze příkazů

Druhým prvkem databáze jsou dvě tabulky představující stromovou strukturu dostupných konfiguračních cest *dirs*, respektive dostupných příkazů *cmds*. Práci s těmito tabulkami zajišťuje třída *Database*. Třída zajišťuje komunikaci, výběry, vkládání dat a kopírování databází sloužící k vytváření nových konfiguračních stromů, viz Pohledy. Tabulky opět nejsou nikterak složité, objevuje se zde také relace mezi



Obr. 4.3: Diagram realizace databázových tabulek obsahujících záznamy konfiguračních cest a příkazů

nimi propojující dané umístění v konfiguračním stromu se sadou dostupných vybraných příkazů.

Tabulka *dirs* obsahuje:

- *id* - jednoznačný číselný automaticky generovaný identifikátor sloužící jako primární klíč,
- *id* - číselné označení nadřazeného bodu v konfiguračním stromu,
- *dir* - hodnota typu VARCHAR uchovávající název aktuálního bodu v konfiguračním stromu.

Tabulka *cmds* obsahuje:

- *id* - jednoznačný číselný automaticky generovaný identifikátor sloužící jako primární klíč,
- *add_cmd* - číselná hodnota uchovávající informaci o dostupnosti daného příkazu v daném bodě konfiguračního stromu,
- $\vdots$

- *comment_cmd* - číselná hodnota uchovávající informaci o dostupnosti daného příkazu v daném bodě konfiguračního stromu,
- *dir* - hodnota typu VARCHAR uchovávající název aktuálního bodu v konfiguračním stromu.

4.3 GUI

Grafické rozhraní bylo implementováno pomocí knihovny DearPyGui [17]. Toto grafické rozhraní bylo vybráno především kvůli schopnosti pracovat se systémem okno v okně, což byl jeden ze základních předpokladů pro realizaci této práce. Jako další výhodou, ačkoli pro tuto práci ne tak podstatnou, je podpora hardwarové akcelerace. Práce s tímto rozhraním není nikterak složitá, a co se struktury kódu týče, je silně benevolentní.

Výpis 4.2: Ukázka inicializace konfiguračního okna, zkráceno

```

1 def __init__(self, cmdDbPath="", devs=""):
2     """Initialises configui object"""
3     # vytvoření kontextu
4     dpg.create_context()
5     ...
6     # volání funkce pro vytvoření hlavního okna
7     MainWindow, name = self.openMainWindow()
8     """Opens main configuration window"""
9
10    # nastavení základních parametrů a inicializace okna
11    dpg.create_viewport(title='Micoto - configure '+name,
12                        width=1500, height=1000)
13    dpg.setup_dearpygui()
14    dpg.show_viewport()
15    dpg.set_primary_window(MainWindow, True)
16    dpg.start_dearpygui()
17    dpg.destroy_context()
18
19    def openMainWindow(self):
20        """Opens main configuration window"""
21        # vytvoření okna
22        with dpg.window(label="Main", tag="Main", on_close=
23                      lambda: dpg.delete_item(MainWindow)) as MainWindow:
24            # definice horizontálního menu

```

```

23     with dpg.menu_bar():
24         # přidání položky do menu "Theme"
25         with dpg.menu(label="Theme"):
26             EditThemePlugin()
27             dpg.add_menu_item(label="Fonts", callback=lambda:
                dpg.show_font_manager())
28     ...
29     # vytvoření skupiny jejíž prvky budou zarovnány
        horizontálně
30     with dpg.group(horizontal=False, parent=MainWindow,
        tag="Menu", width=100, height=dpg.get_item_height(
        MainWindow)):
31         # vytvoření tabulky
32         with dpg.table(header_row=False):
33             dpg.add_table_column()
34             # smyčka vytvářející jednotlivá tlačítka
35             for dirId in self.middle.getDirsWithoutParent():
36                 with dpg.table_row():
37                     user_data={"pos":0, "dirId":dirId}
38                     dpg.add_button(label=self.middle.getDirName(
                        dirId), user_data=user_data, callback=self
                        .openDirWindow)
39     name=", ".join(list(self.devices.keys()))
40     return MainWindow, name

```

Z ukázky je zřejmé, že vytvoření okna a přidávání jednotlivých grafických elementů je poměrně jednoduché a logické. Díky možnosti integrace do kterékoli části kódu se s touto knihovnou pracuje opravdu příjemně a intuitivně.

5 Testování

Již během vývoje byla aplikace testována všemi možnými různými způsoby, a to především ve smyslu náhodné (nelogické) interakce uživatele s prostředím aplikace. V pozdějších fázích práce došlo i na ošetřování výjimek ve smyslu odpojení zařízení či výběru špatného databázového souboru. Aplikace byla testována na vícero operačních systémech. Na všech operačních systémech aplikace běžela bez sebemenších problémů po celou dobu testování. Výčtem se jednalo o:

- Microsoft Windows 10 Home, 22H2 (build 19045.6466),
- Manjaro + KDE Plasma,
- Debian GNU/Linux 13.4 (trixie), kernel 7.0.2-2-pve + xfce4.20, X11 (LXC kontejner),
- Linux Mint 22.3, kernel 6.17.0-29-generic, Cinnamon 6.6.7, X11,
- Ubuntu 23.04 (Xubuntu), kernel 6.17.4-2-pve + xfce4.18 (LXC kontejner).

5.1 Ošetřené stavy

Jak bylo zmíněno výše, již během vývoje byla aplikace průběžně testována a později doplněna i o mechanismy řešící krizové situace běhu aplikace.

Práce s databází je ošetřena hned na několika místech. V první řadě dochází při vytváření databáze ke kontrole, zda již dané tabulky či soubor existují. V případě, že ne, dojde automaticky k jejich vytvoření. Pokud tabulky již existují, může dojít k opětovnému zápisu. Tato vlastnost může být výhodná při správném použití, ale dokáže také způsobit nestability, proto ji není vhodné využívat. Jako druhý záchranný bod je kontrola, zda existují příslušné tabulky v databázi. Pro vybranou databázi příkazů se kontroluje existence tabulky *cmds* a *dirs*, pro databázi zařízení existence tabulky *devices*. Tato kontrola by se dala ještě rozšířit na kontrolu předepsané struktury tabulky.

Při přidávání zařízení probíhá kontrola již při zadávání IP adresy, která je i graficky znázorněna. Aktuální verze aplikace byla testována pouze na IPv4, ačkoli implementace podpory IPv6 by neměla být problémem. Jelikož byl požadavek od vedoucího na funkční IPv4, vývoj vedl na funkčnost aplikace s IPv4, IPv6 nebyla dostatečně testována a proto zde není vůbec prezentována.

Hlavním indikátorem funkčnosti rozhraní API na zařízení a správnosti zadaných přihlašovacích údajů je položka *name* v hlavní aplikaci. Pokud nedojde k zobrazení položky *name*, spojení ze zařízením se při vytváření nepovedlo sestavit. Tato zdánlivá chyba je zde úmyslně. Ne nutně musí vždy platit, že administrátor bude chtít přidávat aktuálně dosažitelné zařízení s funkčním rozhraním API. Může se také jednat pouze o přípravu databáze zařízení určených do jiné, aktuálně nedosažitelné,

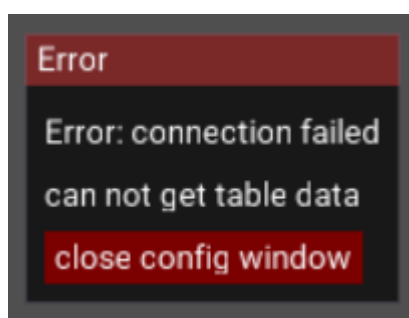
sítě. Bohužel v tuto chvíli není implementován mechanismus rozlišení s indikací typu chyby a ani možnost, v případě dostupnosti zařízení, dané jméno zařízení manuálně či automaticky doplnit.

Při pokusu o připojení a otevření konfiguračního okna je prováděna kontrola spojení. Pokud se nepodařilo navázat spojení alespoň s jedním se zařízení, nedojde k otevření konfigurační aplikace a je zobrazeno okno s hláškou, že se nepodařilo otevřít konfigurační aplikaci. I zde je prostor k vylepšení, a to indikace, s jakým zařízením se nepodařilo spojit či co znemožnilo otevření spojení.

Pokud dojde ke znemožnění komunikace se zařízením během provádění konfigurace, je zobrazeno v konfiguračním okně upozornění, případně i s bližším popisem, jakou operaci se nepodařilo provést. V rámci okna upozornění je také zobrazeno tlačítko sloužící k uzavření konfiguračního okna. Prostor pro vylepšení spočívá zejména v informování uživatele, které ze zařízení není k dispozici, případně upravení celého systému konfiguračního okna tak, aby nebylo nutné konfigurační okno zavřít, například informováním uživatele a upravením seznamu konfigurovaných zařízení.

Je také ošetřeno okno *treeview*. Při výběru databázového souboru, který neobsahuje sadu tabulek uchováající strukturu příkazů, viz výše, nedojde k otevření okna, nýbrž k zobrazení upozornění, že okno *treeview* nešlo otevřít. Opět by se dalo toto chování vylepšit, a to přesnějším popisem důvodu selhání otevření tohoto okna. Co se ukládání vybraných příkazů do souboru týče, při volbě již existujícího souboru dojde k jeho kompletnímu přepsání. Určitě by bylo vhodné fungování tříd *Database* a *DBConn* v takovéto situaci sjednotit.

Aplikace by zajisté potřebovala vylepšit způsob zpracování výjimek, každopádně během testování v aktuálním stavu se autoru ani nikomu z oslovených nepodařilo způsobit pád aplikace, což lze považovat za dobrý výsledek.



Obr. 5.1: Ukázka chybového okna

6 Struktura aplikace

Aplikace je rozdělena do několika samostatně funkčních celků, a to administrátorské aplikace, samotné konfigurační aplikace a úvodní aplikace, která je otevřena ihned po spuštění aplikace micoto.

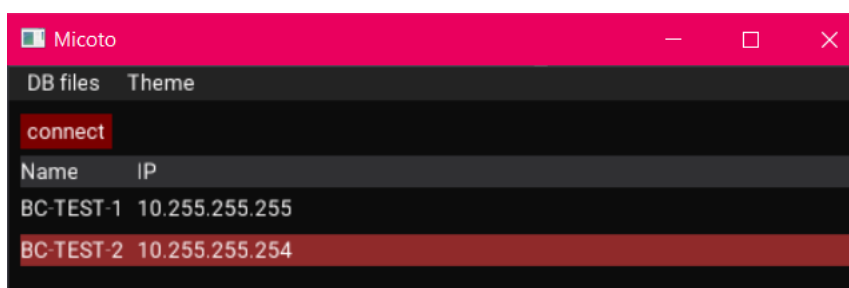
Co se databází týče, je aplikace rozdělena na dvě části. Tyto části jsou databáze s příkazy a databáze s přihlašovacími údaji. Toto rozdělení má své výhody i nevýhody.

Pro dané rozdělení databází bylo rozhodnuto kvůli praktičnosti reálného nasazení. Administrátorovi stačí pro změnu dostupných příkazů či změnu dostupných zařízení roz distribuovat nové databázové soubory s definovanou strukturou, a to nezávisle na ostatních uživateli. Tato skutečnost také umožňuje tyto dvě databáze upravovat nezávisle na sobě a tvořit jejich kombinace.

6.1 Úvodní aplikace

Tato aplikace je základním prvkem pro nastavení a ovládání aplikace. Nabízí jednoduché uživatelské rozhraní.

Mezi prvotní úkony v této aplikaci je výběr databázového souboru obsahujícího přístupové údaje ke spravovaným zařízením a zadáním hesla pro rozšifrování těchto údajů. Jako další krok je výběr a nastavení databázového souboru obsahujícího sadu dostupných příkazů. Následně lze údaje rozšifrovat. Dostupná zařízení z rozšifrované databáze jsou zobrazena v okně. Kliknutím na daný záznam dojde k jeho označení. Po výběru požadovaných zařízení lze stisknutím tlačítka *connect* provést spojení s danými zařízeními. Dojde k otevření nového okna, a to okna aplikace sloužící ke konfiguraci dříve vybraných zařízení.



Obr. 6.1: Grafické rozhraní hlavní aplikace

6.1.1 Vnitřní struktura

Aplikace se spouští buďto příkazem či poklepáním na soubor. Pokud chce daný uživatel spustit aplikaci, není třeba aplikaci spouštět s parametry. Aplikaci lze také spustit v administrátorském módu, pro jehož spuštění je třeba vytvořit objekt typu *GUI* s parametrem *admin=True*. Nastavení hodnoty parametru *admin* na *True* způsobí zobrazení jinak skrytých grafických prvků, kterými lze aktivovat administrátorské části kódu, které jsou jinak uživateli skryty.

Třída *db_crypto*

Tato třída reprezentuje a realizuje operace spojené s vytvořením a úpravami databáze zařízení. Společně s těmito operacemi také zodpovídá za šifrování a dešifrování přístupových údajů. Mimo jiné využívá knihovnu *cryptography*.

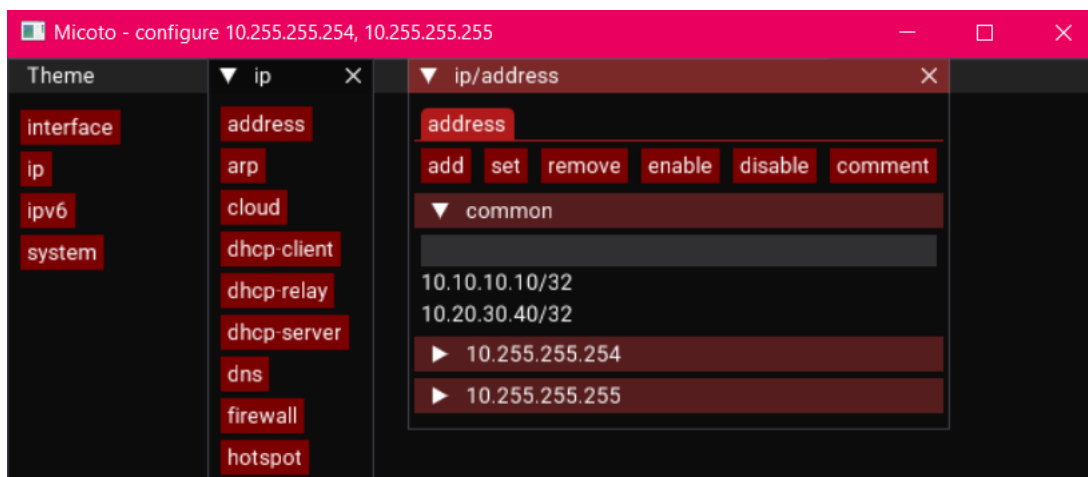
Výpis 6.1: Metoda objektu *db_crypto* (6.1.1) pro zašifrování a uložení přístupových údajů

```
1 def insert(self, devIp, devUser, devPass):
2     """Inserts new device to device database, catches and
3     saves system name"""
4     # ověří, zda zařízení se stejnou IP již není v databázi
5     if self.checkExistance(devIp):
6         return False
7     #devHost
8     # pokusí se navázat spojení se zařízením pro získání
9     hostname
10    identity = apiros.getResponse(devIp, devUser, devPass,
11    "/system/identity/print")
12    if identity:
13        devHost = identity["name"]
14    else:
15        devHost = ""
16    #db insert
17    # pošle vložené heslo do funkce vracející zašifrované
18    heslo (ciphertext) a iv (initialization vector)
19    devCipher, devIv = self.encrypt(devPass.encode().
20    ljust(32)[:32])
21    # uložení dat do databáze
22    self.cur.execute("INSERT INTO devices (devHost, devIp,
23    devUser, devPass, devIv) VALUES (?, ?, ?, ?, ?)", (
24    devHost, devIp, devUser, devCipher, devIv))
25    self.con.commit()
26    return True
```

6.2 Konfigurační aplikace

Po inicializaci a připojení dle 6.1 k vybraným zařízením dojde k otevření okna konfigurační aplikace.

Na levé straně okna aplikace nabízí dostupné kořenové volby (příkazy), které má uživatel k dispozici. Klinutím na ně je otevřeno další vnitřní okno, které nabízí výběr možných konfigurací a/nebo výpis již nastavených/dostupných prvků, na které je možné dané příkazy aplikovat.



Obr. 6.2: Grafické rozhraní konfigurační aplikace

Na obrázku 6.2 lze spatřit část okna s názvem *common*. Právě tato část je tou částí, kvůli které celá práce vznikla. Nachází se v ní společné prvky alespoň dvou zařízení, ke kterým je aktuálně konfigurační aplikace připojena. Výčet těchto zařízení se nachází v názvu okna, v tomto případě se jedná o zařízení s IP adresami 10.255.255.254 a 10.255.255.254.

Po přesunu kurzoru nad daný záznam je zobrazeno informační vyskakovací pole, ve kterém jsou uvedeny detaily daného záznamu pro dané zařízení. V tomto případě se jedná o IP adresu a informace k jejímu záznamu patřící, viz 6.3.

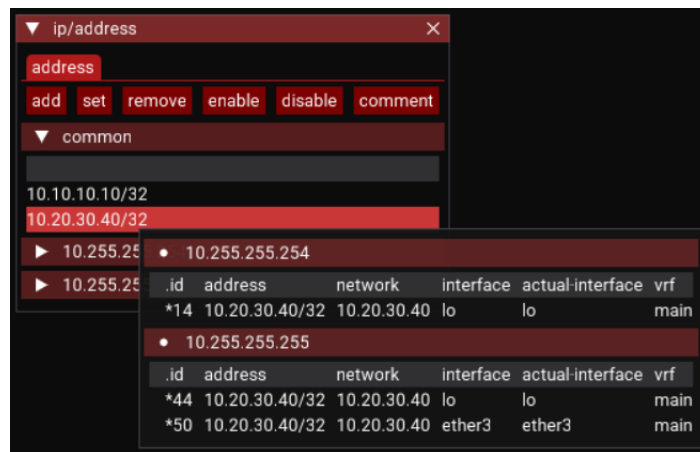
Pro zobrazení detailu všech nastavených parametrů pro záznamy stejného typu se nachází pod sekci *common* rolovací tabulka nesoucí IP adresu daného zařízení jako název. Po kliknutí na daný záznam je zobrazena daná tabulka 6.4.

Práce s vybranými záznamy probíhá za použití tlačítek v horní části vnitřního konfiguračního okna. Po označení požadovaných záznamů a kliknutí na tlačítko s požadovanou akcí je zobrazeno konfigurační podokno se všemi možnostmi úprav. Pole, která mají u pravé strany šipku, po rozkliknutí zobrazí roletové menu, ve kterém se nacházejí dostupné možnosti konfigurace. Ostatní položky jsou text boxy určené k manuálnímu zadání hodnoty parametru, viz 6.5.

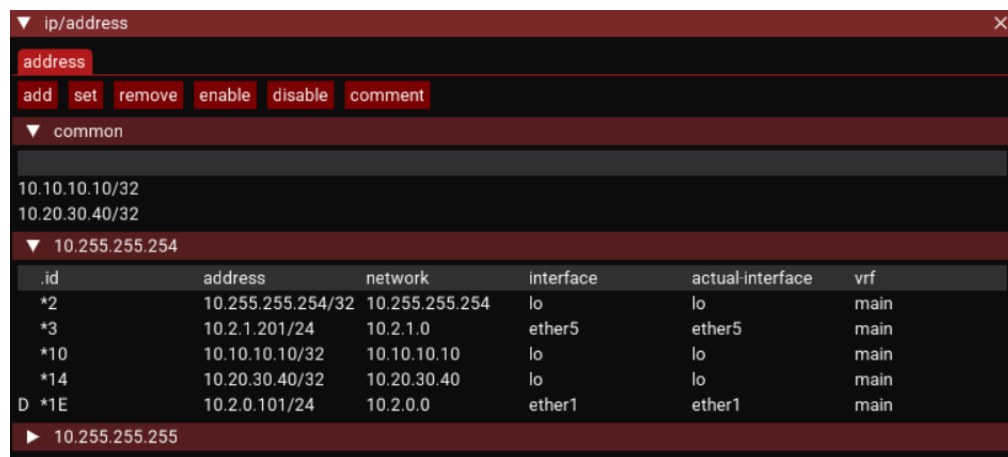
Položka *numbers* zobrazuje seznam dříve vybraných zařízení pro konfiguraci včetně *id* jednotlivých upravovaných položek.

Položka *test* je výstupem z konfigurovaných zařízení. Jedná se o indikátor správnosti zadaných parametrů. Kontrola probíhá až po kliknutí na tlačítko *apply*.

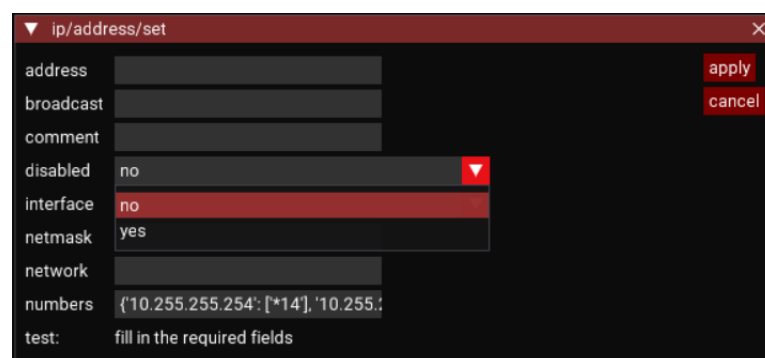
Po stisku tlačítka *apply* dojde k postupnému navázání spojení s vybranými zařízeními a provedení konfiguračních změn.



Obr. 6.3: Detail grafického rozhraní konfigurační aplikace - sekce *common*



Obr. 6.4: Detail grafického rozhraní konfigurační aplikace - záznamy vybraného zařízení



Obr. 6.5: Detail grafického rozhraní konfigurační aplikace - konfigurační podokno

6.2.1 Vnitřní struktura

Aplikace je spouštěna jako podproces úvodní aplikace. Spouštění je provedeno pomocí příkazu *py*, za kterým následuje cesta k souboru aplikace *conf_gui.py*. Jako další spouštěcí parametr je cesta k databázovému souboru s příkazy. Jako poslední je slovník ve formě vnořených slovníků *{ip_adresa:{uživatelské_jméno:heslo}}*. Je tedy zřejmé a bylo to také testováno, že se jedná o samostatně spustitelnou aplikaci.

Třída *device*

Jedná se o třídu reprezentující jednotlivá zařízení. Mezi její základní vlastnosti patří například objekt *ApiRos* zprostředkávající surovou komunikaci na úrovni socketu či objekt *Api* sloužící ke zprostředkování samotné komunikace pomocí prostého volání funkcí.

Třída *middleware*

Ihned po spuštění aplikace *conf_gui* (6.2) dochází k vytvoření instance objektu *middleware*, který slouží jakožto adaptační vrstva mezi grafickým rozhraním, databází a zařízeními. Tvoří tedy jednotný bod pro komunikaci se zařízeními.

Ihned při inicializaci objektu *middleware* dojde k vytvoření objektu třídy *device* (6.2.1) pro každé zařízení, které se nacházelo v parametrech příkazu při spuštění. Třída *middleware* udržuje seznam objektů typu *device* po celou dobu běhu aplikace a vykonává nad nimi požadované operace. Tato třída je také zodpovědná za třídění společných prvků vícero zařízení či provedení konfiguračních změn napříč vybranými zařízeními (6.2).

Existence této třídy zajišťuje nezávislost přístupu a práce se zařízeními. Díky tomuto faktu je možné po úpravách vytvořit například webovou aplikaci, u které by konfigurace probíhala v okně webového prohlížeče a komunikaci se zařízeními by zajišťoval centrální server, který by mohl být snadno obohacen o další funkce, jako je například integrace adresářových služeb či snazší správa zpřístupněných uživatelských příkazů.

Výpis 6.2: Metoda objektu *middleware* (6.2.1) sloužící k provedení konfiguračních změn

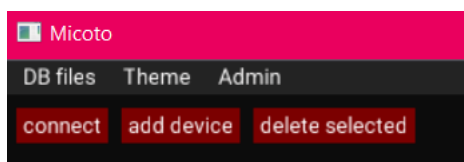
```
1 def applyToDevices(self, argVals="", dirId="", cmdName=""
  , spacer="/", selected=False):
2     """Checks values and applies changes"""
3     #vypíše úplnou cestu příkazu
4     pathDef=spacer+self.printDirPath(dirId=dirId, spacer=
      spacer)+spacer+cmdName
5     msgs=dict()
6     #postupně prochází slovník zařízení a změn, které se
      mají provést
7     for devIp, ids in selected.items():
8         # najde správné zařízení v poli zařízení
9         for r in self.devList:
10             if r.getDevIp()==devIp:
11                 dev=r
12                 break
13         argVals=argVals.copy()
14         # řeší problém vzniklý různými způsoby výběru
            konfigurovaných zařízení
15         if "interface" not in argVals.keys():
16             argVals["numbers"]=",".join(ids)
17         try:
18             msg = dev.api.checkValues(argVals=argVals, pathDef=
              pathDef) # provádí samotnou konfiguraci
19         except:
20             raise
21         if msg and "message" in msg:
22             msgs[devIp]=msg["message"]
23         else:
24             msgs[devIp]="ok"
25     # vrací výsledek všech operací (hláška "ok" či chybu)
26     return msgs
```

Třída window

Jedná se o třídu reprezentující okno v programu. Slouží k uchování informací o okně, jako je například jeho název, aktuální id adresáře ve struktuře příkazů a udržuje seznam označených položek.

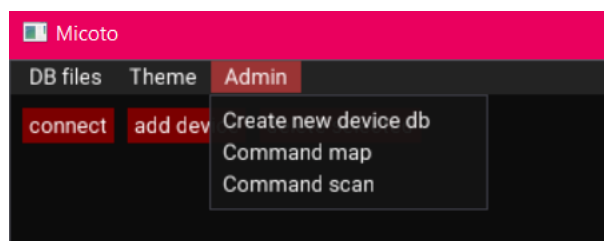
6.3 Administrátorská aplikace

Při spuštění aplikace *micoto* pomocí skriptu *admin* dojde ke spuštění aplikace v administrátorském módu. Tento mód umožňuje úpravy databází, skenování dostupných příkazů či jejich úpravu. Na obrázku je možné spatřit již první změny, a to



Obr. 6.6: Detail grafického rozhraní aplikace - administrátorský mód

zpřístupnění nabídky *Admin* a dvou nových tlačítek sloužících k úpravě databáze zařízení. Při kliknutí na tlačítko *Admin* v horní části obrázku je zobrazen rolovací menu s možnostmi úprav. Nabídka menu obsahuje pouze tři, ale zato velice důle-



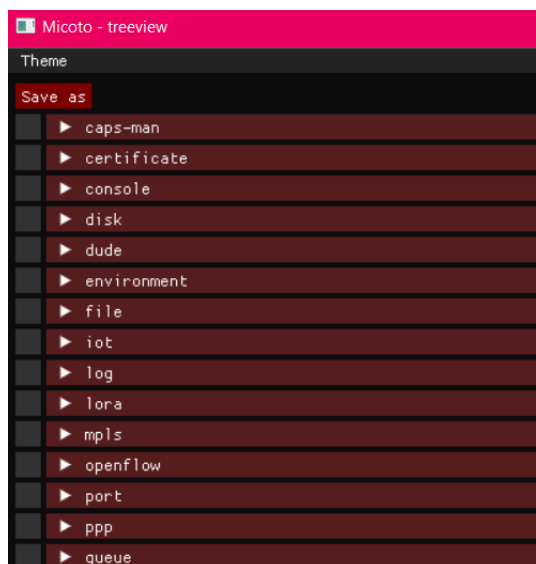
Obr. 6.7: Detail grafického rozhraní aplikace - rozvinuté administrátorské menu

žité, možnosti. Možnost *Create new device db* vyzve uživatele k výběru umístění, následně jména a hesla databáze zařízení. Výsledkem je vytvoření prázdné databáze zařízení společně s nastavením šifrovacího hesla.

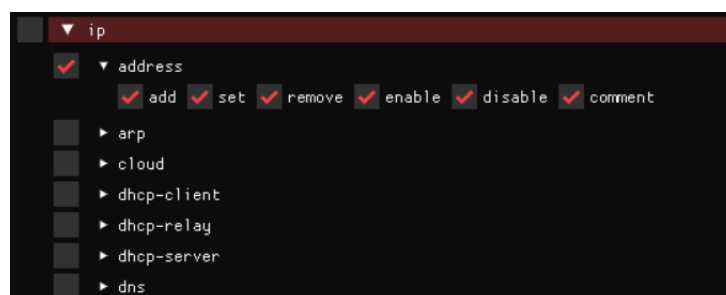
Další z možností je *Command scan*. Tato možnost slouží k naskenování příkazové struktury z vybraného zařízení. Před aktivací je tedy nutné do databáze zařízení přidat alespoň jedno zařízení a to označit.

Poslední z možností je *Command map*. Po kliknutí na ni je uživatel vyzván pro výběr databáze s naskenovanými příkazy. Je doporučeno vybrat databázi obsahující všechny dostupné příkazy. Po výběru je otevřena aplikace *Micoto - treeview* jako podproces úvodní aplikace.

Okno aplikace *treeview* zobrazuje stromovou strukturu příkazů vybrané databáze. Pomocí kliknutí na řádek se lze zanořit do stromu příkazů dále od kořene. Výběrem zaškrtnutí políčka na levé straně dané větve je možné vybrat všechny příkazy v dané větvi. Na konci větví se vždy nachází možnost výběru daných možností úprav daného příkazu.



Obr. 6.8: Detail grafického rozhraní aplikace - okno *treeview*



Obr. 6.9: Detail grafického rozhraní aplikace - okno *treeview* s otevřenými možnostmi výběru větve

Pomocí tlačítka *Save as* a zadáním názvu souboru je možné aktuálně vybrané příkazy uložit do nové databáze příkazů.

Závěr

Cílem této bakalářské práce bylo navrhnout a naprogramovat multiplatformní aplikaci v programovacím jazyce Python sloužící k hromadným konfiguracím zařízení s operačním systémem RouterOS prostřednictvím oficiálního zabezpečeného API rozhraní na portu 8729.

Zadání se podařilo ve stanoveném rozsahu splnit. Aplikace je ve funkčním stavu a lze z ní provádět veškeré úpravy konfigurace. Hromadné provádění změn je realizováno nikoli pomocí šablon, ale za pomoci vlastního konfiguračního okna pracujícího s graficky reprezentovanou formou stromu příkazů systému RouterOS.

Po řešerši na téma správy logů a konzultaci s vedoucím bylo rozhodnuto, že tuto část není třeba realizovat, jelikož existuje spousta veřejně dostupných systémů řešících tuto problematiku. Příkladem může být systém Zabbix [20] či LibreNMS [21]. Pro autora to byla velká zkušenost, jednalo se totiž o jeho první projekt takového rozsahu.

Celý kód je dostupný na platformě GitHub pod licencí MIT – kód, dokumentace, návod.

Literatura

- [1] IEEE. *The world's largest technical professional organization dedicated to advancing technology for the benefit of humanity*. Online. <https://www.ieee.org>. [cit. 2026-03-04].
- [2] Cisco Systems *AI Infrastructure, Secure Networking, and Software Solutions* Online. <https://www.cisco.com/site/us/en/index.html>. [cit. 2026-03-04].
- [3] HP *Laptop Computers, Desktops, Printers, Ink & Toner / HP® Official Site* Online. <https://www.hp.com/us-en/home.html>. [cit. 2026-03-04].
- [4] HP Enterprise *Hewlett Packard Enterprise (HPE) / Czechia* Online. <https://www.hpe.com/cz/en/home.html>. [cit. 2026-03-04].
- [5] Arista Networks *Data-Driven Cloud Networking - Arista* Online. <https://www.arista.com/en/>. [cit. 2026-03-04].
- [6] Ubiquiti Networks *Ubiquiti - Rethinking IT - Ubiquiti* Online. <https://ui.com/>. [cit. 2026-03-04].
- [7] SIA Mikrotiks *MikroTik Routers and Wireless* Online. <https://mikrotik.com/>. [cit. 2026-03-04].
- [8] MikroTik Routers and Wireless *Products* Online. <https://mikrotik.com/products>. [cit. 2026-03-04].
- [9] MikroTik Routers and Wireless *Software* Online. <https://mikrotik.com/download>. [cit. 2026-03-04].
- [10] X.200 : Information technology - Open Systems Interconnection *Basic Reference Model: The basic model* Online. <https://www.itu.int/rec/T-REC-X.200/en/>. [cit. 2026-03-04].
- [11] RouterOS - MikroTik Documentation *Python3 Example* Online. <https://help.mikrotik.com/docs/spaces/ROS/pages/47579209/Python3+Example>. [cit. 2026-03-04].
- [12] SQLite *Home Page* Online. <https://sqlite.org/>. [cit. 2026-03-04].
- [13] sqlite3 — DB-API 2.0 interface for SQLite databases *Python 3.14.0 documentation* Online. <https://docs.python.org/3/library/sqlite3.html>. [cit. 2026-03-04].

- [14] Fernet (symmetric encryption) *Cryptography 47.0.0.dev1 documentation* Online. <https://cryptography.io/en/latest/fernet/>. [cit. 2026-03-04].
- [15] Národní úřad pro kybernetickou a informační bezpečnost *Doporučení v oblasti kryptografických prostředků* Online. https://nukib.gov.cz/download/uredni_deska/Minimalni_pozadavky_v4_FINAL.pdf. [cit. 2026-03-04].
- [16] GuiProgramming - Python Wiki *GUI Programming in Python* Online. <https://wiki.python.org/moin/GuiProgramming>. [cit. 2026-03-04].
- [17] GitHub - hoffstadt/DearPyGui *Dear PyGui: A fast and powerful Graphical User Interface Toolkit for Python with minimal dependencies* Online. <https://github.com/hoffstadt/DearPyGui>. [cit. 2026-03-04].
- [18] Dear PyGui's Documentation *Dear PyGui documentation* Online. <https://dearpygui.readthedocs.io/en/latest/index.html>. [cit. 2026-03-04].
- [19] Cisco Systems *Role-Based CLI Access* Online. https://www.cisco.com/en/US/docs/ios/12_3t/12_3t7/feature/guide/gtclivws.html. [cit. 2026-03-04].
- [20] Zabbix *The enterprise-class open source observability solution* Online. <https://www.zabbix.com>. [cit. 2026-05-25].
- [21] LibreNMS Online. <https://www.librenms.org>. [cit. 2026-05-25].

Seznam symbolů a zkratek

CHR	Cloud Hosted Router
GUI	grafické uživatelské rozhraní (Graphical User Interface)
ROS	Router Operating System

A Aplikace micoto

Obsahem elektronické přílohy jsou zdrojové kódy aplikace micoto včetně dokumentačních souborů. Z výpisu je záměrně vynechána struktura adresáře *docs*, jelikož není pro tuto práci důležitá kromě souboru *index.html*. Zobrazením daného souboru je možné procházet lokální verzi dokumentace kódu.

```
micoto
├── ChooseFontsPlugin.py
├── app
│   ├── EditThemePlugin.py
│   ├── api
│   │   ├── api.py
│   │   └── apiros.py
│   ├── command_scan.py
│   ├── conf_gui.py
│   ├── db
│   │   ├── db.py
│   │   └── db_crypto.py
│   ├── device.py
│   ├── middleware.py
│   ├── themes
│   │   └── default.ini
│   ├── treeview.py
│   └── window.py
├── docs
│   ├── doc_conf
│   ├── ...
│   └── index.html
├── micoto.py
└── themes
    ├── OpenSans.ttf
    ├── Roboto.ttf
    └── default.ini
```